

FluxMoE: Decoupling Expert Residency for High-Performance MoE Serving

Qingxiu Liu¹, Yongchao He^{2*}, Runhan Jiang², Zion Wang³, Bohan Zhao², Mi Zhang⁴,

Patrick P. C. Lee¹

¹CUHK ²Unaffiliated ³SCITIX ⁴ICT, CAS

Abstract

Mixture-of-Experts (MoE) models have become mainstream for scaling language models to hundreds of billions of expert parameters. Despite sparse expert activation, existing inference engines keep all experts GPU-resident, crowding out the key-value cache in large-batch, long-output offline workloads. We present FluxMoE, which decouples experts from physical GPU residency and adapts their footprint to available memory through a new *expert paging* abstraction. FluxMoE combines PagedTensor for transparent remapping, a bandwidth-balanced hierarchy spanning losslessly compressed GPU memory and host DRAM, and a budget-aware residency planner. Unlike CPU-GPU co-inference and whole-layer offloading, FluxMoE streams weights on demand while keeping expert computation on GPUs. We implement FluxMoE atop vLLM and evaluate it on three MoE models. For GLM-4.5 on 8×H20 GPUs, FluxMoE delivers up to 7.2× vLLM’s throughput and 79.0% lower average Time-Per-Output-Token (TPOT), without measurable model-quality loss using lossless compression. For Mixtral-8×7B-Instruct on 2×L40S GPUs, where weight-resident vLLM cannot fit, FluxMoE delivers 4.3× KTransformers’s throughput and 29.1% lower average TPOT.

1 Introduction

Over the past year, Mixture-of-Experts (MoE) architectures have become mainstream for scaling LLMs without proportionally increasing per-token computation. This gain comes with unprecedented weight footprints: leading open-weight MoEs span hundreds of billions to one trillion parameters [8, 34, 35]. Conventional inference engines [22, 47] keep all weights GPU-resident, crowding out the key-value (KV) cache, which retains the prior-token attention keys and values. Since KV demand grows with active sequence count and cached length, reduced capacity forces the scheduler to process fewer sequences per decoding iteration, lowering throughput. This pressure is acute in throughput-oriented offline workloads such as synthetic-data generation [37], post-training rollout generation [31], and model evaluation [5], where large batches and long outputs increase KV demand. Adding GPUs relieves this pressure but can reduce per-GPU efficiency. On our H20 testbed, scaling a vLLM [22] deployment of Qwen3-Next-80B-A3B-Instruct [40] from two

to eight GPUs via model parallelism at batch size 256 with 4,096-token outputs raises sequence-state headroom per GPU from 10.1 GB to 69.8 GB but reduces per-GPU throughput by 62.4%. Large batches may activate most experts within each layer (Appendix A), yet the layers execute sequentially.

Must the experts of all layers remain GPU-resident simultaneously? Our key insight is to stream expert weights while retaining the non-expert backbone. Since expert pools constitute over 95% of the model-weight footprint (§2.1), this makes most weight memory eligible for reclamation; keeping the non-expert backbone resident lets its kernels run without weight materialization, creating an overlap window for materializing the next expert set (§2). This is feasible because, although tokens may activate different experts within a layer, they preserve the fixed, layer-by-layer execution order. Thus, compute kernels access an expert’s weights only while executing its layer; otherwise, those weights occupy GPU memory without contributing to computation. Expert weights can be managed as *streamed weights*, materialized before use and released afterward, rather than retained as persistent GPU state by conventional engines [22, 47]. This leads to a new streaming execution model for MoE inference:

$$\text{model} = \text{compute graph} + \text{streamed weights}. \quad (1)$$

The compute graph is static and follows a fixed layer order, while streaming supplies the current layer’s expert weights. We formalize this model as *expert paging*, which decouples experts from persistent GPU residency and frees GPU memory for the KV cache and other runtime state.

Realizing expert paging poses three challenges. (i) *Transparency*: expert weights must remain accessible to existing kernels as their physical residency changes, without kernel changes. (ii) *Timely materialization*: expert weights must be materialized fast enough to reduce inference stalls; streaming expert weights from host DRAM over PCIe may not finish before the layer executes. (iii) *Adaptability*: expert residency must adapt to changing KV-cache memory pressure without incurring excessive materialization overhead.

We design FluxMoE, a streaming execution model that augments existing MoE inference frameworks with dynamic expert materialization through three tightly integrated mechanisms. (i) The *PagedTensor* (§4.1) addresses transparency by reserving stable virtual addresses for all experts and mapping them to dedicated or reusable physical blocks, without modifying PyTorch operators [27] or Triton kernels [36]. (ii) The

*Corresponding author.

expert memory hierarchy (§4.2) addresses timely materialization by retaining a portion of expert weights in losslessly compressed form in GPU memory, placing the remainder in host DRAM, and balancing GPU decompression and PCIe transfers according to their bandwidths. (iii) The *budget-aware residency planner* (§4.3) selects expert residency via scheduler state and materialization measurements under changing KV demand.

We implement FluxMoE atop vLLM (v0.23.0) [22] and evaluate it using three MoE models [20, 34, 40]. On 8×H20 GPUs, FluxMoE achieves up to 7.2× vLLM’s throughput for GLM-4.5 [34] and reduces average Time-Per-Output-Token (TPOT) by 79.0% without measurable quality loss. On 2×L40S GPUs, vLLM cannot deploy Mixtral-8×7B-Instruct [20]; FluxMoE serves it with 4.3× KTransformers’s [7] throughput and 29.1% lower average TPOT. We make three contributions:

- We introduce *expert paging*, a streaming abstraction that decouples experts from persistent GPU residency and reclaims memory for the KV cache and other runtime state.
- We design FluxMoE with PagedTensor for transparent tensor remapping, a bandwidth-balanced expert hierarchy, and a budget-aware residency planner.
- We implement FluxMoE atop vLLM [22] and evaluate it on three MoE models [20, 34, 40] spanning 47B–355B parameters across two NVIDIA GPU platforms.

2 MoE Inference Memory Characterization

Under a fixed GPU-memory budget, model weights compete with the KV cache. We first identify that expert pools dominate reclaimable weight memory, then show when more KV capacity raises throughput. These findings motivate expert-only streaming and adaptive expert residency.

2.1 Expert Weights Dominate Reclaimable Memory

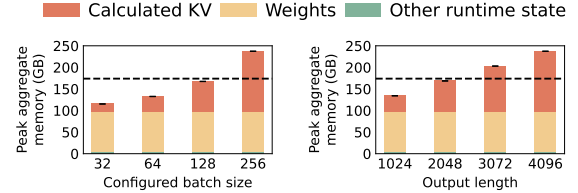
MoEs scale capacity with large expert pools. Sparse activation limits per-token computation but not memory use since conventional engines keep all experts GPU-resident [22, 47].

We quantify this memory pressure by running Mixtral-8×7B-Instruct [20] with vLLM [22] on 4×L40S GPUs (§6.1). Its weights occupy 94 GB of the 173.9 GB aggregate GPU budget, leaving 79.9 GB for the KV cache and other runtime state. We estimate peak KV demand as total prompt-plus-output tokens times the model’s BFloat16 KV bytes per token, excluding padding. Figure 1 shows that a batch of 256 with 4,096-token outputs requires 139.3 GB of KV storage, exceeding this headroom by 59.4 GB. At this batch size, extending the output from 3,072 to 4,096 adds 34.4 GB. Thus, a model may fit in GPU memory while its target workload does not.

Expert weights constitute 95.2–96.3% of evaluated models’ parameters (Table 1), making nearly all weight memory eligible for reclamation. Retaining the non-expert backbone avoids loading it and lets its computation overlap expert loading. Prior MoE offloading systems also overlap expert

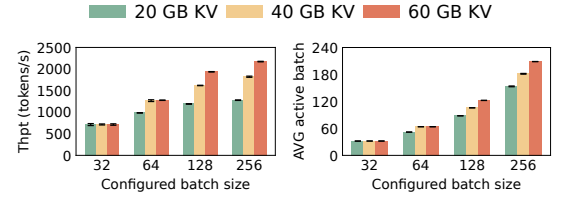
Table 1. Total and expert parameters of the evaluated MoEs.

Model	Total params (B)	Expert params (B) (% of total params)
GLM-4.5 [34]	355	338 (95.2%)
Qwen3-Next-80B-A3B-Instruct [40]	80	77 (96.3%)
Mixtral-8×7B-Instruct [20]	47	45 (95.7%)



(a) Output length = 4,096 (b) Configured batch size = 256

Figure 1. Calculated peak memory demand for Mixtral-8×7B-Instruct [20]; dashed line: 4×L40S memory budget.



(a) Throughput (b) Average active batch size

Figure 2. Effect of aggregate GPU KV-cache capacity on Mixtral-8×7B-Instruct [20] at an output length of 4,096.

transfers with computation [6, 10, 39]. Whole-layer streaming would reclaim less than 5% additional weight memory while adding non-expert traffic and shortening the loading overlap window. Thus, we stream only expert pools (§3).

2.2 KV-Cache Capacity Constrains Throughput

We distinguish the *configured batch size* B_{cfg} , the number of submitted sequences, from the *active batch size* B_{act} , the number scheduled in one decoding iteration; $\bar{B}_{\text{act}}$ is its average over the run. Each active sequence requires GPU-resident KV blocks, so insufficient KV capacity forces vLLM to defer or preempt requests, making $B_{\text{act}} < B_{\text{cfg}}$. To isolate this effect, we run Mixtral-8×7B-Instruct [20] with vLLM KV offloading on 4×L40S GPUs (§6.1) and vary only aggregate GPU KV capacity. We report throughput and $\bar{B}_{\text{act}}$; GPU memory outside the configured KV budget holds weights and other runtime state or remains unallocated.

Figure 2 reveals two regimes. At $B_{\text{cfg}} = 32$, all configurations sustain $\bar{B}_{\text{act}} \approx B_{\text{cfg}}$; increasing aggregate KV capacity from 20 GB to 60 GB changes both throughput and $\bar{B}_{\text{act}}$ by less than 0.3%. The full submitted batch already fits, so extra KV capacity provides no benefit. At larger B_{cfg} , runs become KV-capacity-bound: smaller KV budgets force the scheduler to defer or preempt more requests, lowering $\bar{B}_{\text{act}}$ and throughput. At $B_{\text{cfg}} = 256$, increasing capacity from 20 GB to 60 GB raises $\bar{B}_{\text{act}}$ by 36.0% and throughput by 70.0%. As each

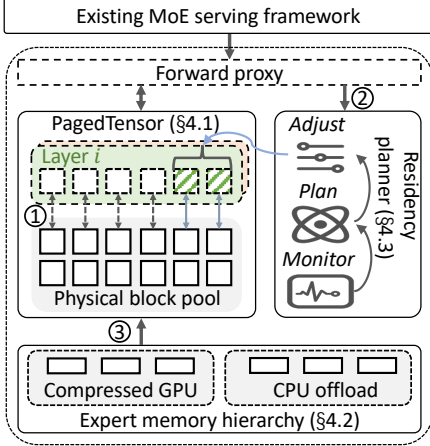


Figure 3. FluxMoE architecture.

active sequence produces one output token per decoding iteration, B_{act} determines the number of tokens produced per iteration and, together with iteration latency, connects KV capacity to throughput.

Together, these observations imply that expert residency should account for KV pressure. Expert pools dominate reclaimable weight memory, but their residency reduces active batch and throughput when KV demand exceeds capacity. Expert paging should therefore release memory under KV pressure but retain more experts otherwise. This motivates on-demand materialization and budget-aware residency (§3).

3 Expert Paging

Expert paging decouples experts from physical GPU residency, treating them as virtual, on-demand resources. Rather than remaining GPU-resident, expert weights are materialized only when their layer is scheduled, consumed by compute kernels, and then evicted to free memory for the KV cache and activation buffers.

We target structurally homogeneous MoEs [20, 34, 40] with N MoE layers in execution order, each with L routed experts of the same architecture; we leave heterogeneous layers to future work. Let E_i^j denote routed expert j in MoE layer i , and $\mathcal{E}_i = \{E_i^1, E_i^2, \dots, E_i^L\}$ denote the routed-expert set, for $i \in \{1, \dots, N\}$ and $j \in \{1, \dots, L\}$. Shared experts, if present, are excluded from $\mathcal{E}_i$. Unless otherwise stated, we use “experts” to refer to routed experts. The model runs under model parallelism across G GPU ranks, indexed by $r \in \{1, \dots, G\}$. We denote $\mathcal{E}_{i,r} \subseteq \mathcal{E}_i$ as rank r ’s expert set for layer i . Under expert parallelism (EP), all ranks divide each layer’s experts evenly into disjoint, consecutive subsets. Under tensor parallelism (TP) alone, $\mathcal{E}_{i,r} = \mathcal{E}_i$, since every rank manages a shard of every expert. By homogeneity, rank r manages $L_r = |\mathcal{E}_{i,r}|$ experts per MoE layer. Unless otherwise noted, all mechanisms in §4 operate on a per-rank basis across the multi-GPU deployment.

During decoding, the N layers execute in order within each iteration, and execution restarts from the first layer in

the next iteration. A *forward proxy* intercepts the model’s forward pass to manage expert materialization. Before the MoE block of layer i begins, the forward proxy ensures that all rank-local experts in $\mathcal{E}_{i,r}$ are materialized and compute-ready. After the MoE block of layer $i-1$ completes, the proxy releases $\mathcal{E}_{i-1,r}$ and materializes $\mathcal{E}_{i+1,r}$ into the reclaimed buffers. Since $\mathcal{E}_{i+1,r}$ is not consumed until the MoE block of layer $i+1$, its materialization window extends from the end of layer $i-1$ ’s MoE block to the beginning of layer $i+1$ ’s MoE block. If materialization does not finish within this window, layer $i+1$ ’s MoE block stalls. For brevity, we use layer i to refer to the entry and exit of its MoE block. Let $R_{i,r}$ denote the rank-local expert set in this sliding window during layer i ’s execution. By the materialization and release rule above, $R_{i,r}$ satisfies:

$$R_{i,r} \subseteq \mathcal{E}_{i,r} \cup \mathcal{E}_{i+1,r}. \quad (2)$$

This window requires two complete rank-local MoE-layer physical blocks, reserving $2/N$ of rank r ’s routed-expert parameter footprint. Beyond this window, FluxMoE uses any available GPU memory to retain additional experts when KV-cache pressure is low, reducing repeated materialization.

4 FluxMoE Design

FluxMoE is an MoE inference middleware that integrates with existing serving frameworks via a lightweight forward proxy (Figure 3). PagedTensor (§4.1) maps dedicated or reusable physical blocks to stable virtual addresses for experts, allowing kernels to access them without modification (①). Under KV pressure, the *budget-aware residency planner* (§4.3) selects which layers remain uncompressed on the GPU and which expert tensors are CPU-offloaded (②). The expert memory hierarchy (§4.2) realizes these decisions across compressed GPU memory and host DRAM, materializing weights into the reusable physical blocks before use (③).

4.1 PagedTensor

On GPU rank r , PagedTensor uses CUDA virtual memory management (VMM) [4] to give rank-local expert-weight tensors stable virtual addresses. Each layer belongs to exactly one of two residency classes: *uncompressed-resident* layers keep their weights in dedicated physical blocks, whereas *pageable* layers materialize expert weights on demand from compressed GPU memory or host DRAM (§4.2) into one of two reusable layer blocks. Residency changes alter only the physical backing, preserving kernel-visible addresses. Unlike vLLM’s PagedAttention [22], which translates request-specific KV-page addresses within kernels, PagedTensor exposes ordinary contiguous tensors, requiring neither kernel changes nor in-kernel address translation.

Formal model. Each expert has two differently sized *weight tensors*: a fused gate/up-projection tensor ($d = 1$) and a down-projection tensor ($d = 2$), which PagedTensor manages separately. For type d on rank r , let $T_{i,r}^{(d)}$ denote layer

i 's contiguous group of rank-local weight tensors ordered by expert index. For homogeneous models (§3), every type- d tensor has byte size $\sigma_r^{(d)}$ across experts and layers; hence, the layer-group size is

$$A_r^{(d)} = L_r \sigma_r^{(d)}. \quad (3)$$

CUDA VMM requires allocation and mapping sizes to be aligned to its minimum granularity g [4]. PagedTensor therefore defines the aligned type- d layer-group size as

$$S_r^{(d)} = \text{alignUp}(A_r^{(d)}, g) = g \left\lceil \frac{A_r^{(d)}}{g} \right\rceil. \quad (4)$$

For each rank r and type d , PagedTensor reserves the g -aligned contiguous virtual region $[v_{0,r}^{(d)}, v_{0,r}^{(d)} + NS_r^{(d)})$ and partitions it into N layer ranges of size $S_r^{(d)}$. For rank-local expert $\ell \in \{1, \dots, L_r\}$, the type- d tensor in layer i has address

$$\text{addr}_{i,r,\ell}^{(d)} = v_{0,r}^{(d)} + (i-1)S_r^{(d)} + (\ell-1)\sigma_r^{(d)}. \quad (5)$$

Given a layer-residency assignment (§4.3), let $\mathcal{U}_r$ and $\mathcal{P}_r$ denote rank r 's uncompressed-resident and pageable layers, where $|\mathcal{U}_r| + |\mathcal{P}_r| = N$. For each type d , PagedTensor maps layer $i \in \mathcal{U}_r$ to a dedicated block $D_{i,r}^{(d)}$ of size $S_r^{(d)}$ and layer $i \in \mathcal{P}_r$ to one of two reusable blocks $F_{r,b_i}^{(d)}$ of size $S_r^{(d)}$, where $b_i = (i-1) \bmod 2$. Only residency changes alter mappings; materialization updates block contents.

CUDA alignment analysis. Since virtual-address reservation consumes no physical GPU memory, type- d alignment overhead on rank r comes from $|\mathcal{U}_r| + 2$ blocks, each padded from $A_r^{(d)}$ to $S_r^{(d)}$:

$$M_{\text{pad},r} = (|\mathcal{U}_r| + 2) \sum_{d=1}^2 (S_r^{(d)} - A_r^{(d)}). \quad (6)$$

All evaluated models [20, 34, 40] satisfy the VMM alignment requirement, so $M_{\text{pad},r} = 0$.

Async materialization. Overlapping reusable-block materialization with inference incurs two ordering constraints:

- *Write-After-Read (WAR)*: Do not overwrite a reusable block until compute kernels reading it complete.
- *Read-After-Write (RAW)*: Do not launch a pageable layer until its weight tensors are materialized.

Algorithm 1 runs independently on each rank. Since each reusable block backs multiple pageable layers, $\text{last}_{b_i,r}$ identifies the layer that last used block $F_{r,b_i}^{(d)}$. Before overwriting it, rank r waits only on its local inference-completion event (WAR), without cross-rank synchronization. Its inference stream likewise waits for local load-completion events before launching the layer (RAW).

4.2 Expert Memory Hierarchy

Pageable layers materialize their routed-expert weight tensors (*pageable tensors*) from compressed GPU memory or host DRAM. FluxMoE balances traffic between these tiers

Algorithm 1 PagedTensor Synchronization Protocol

Input: GPU rank r , pageable layers $\mathcal{P}_r$, tensor groups $T_{i,r}^{(d)}$, inference stream $\text{stream}_{r,\text{infer}}$ and loading-stream sets $\text{streamset}_{i,r,\text{load}}^{(d)}$

Define $b_i = (i-1) \bmod 2$ for each layer i

- 1: **procedure** MATERIALIZELAYER(i)
- 2: **if** $i \in \mathcal{P}_r$ **then**
- 3: **for** $d \in \{1, 2\}$ **do**
- 4: **if** $\text{last}_{b_i,r}$ is defined **then**
- 5: WaitEvent($\text{event}_{\text{last}_{b_i,r},r}^{\text{infer}}, \text{streamset}_{i,r,\text{load}}^{(d)}$) $\triangleright$ WAR
- 6: **end if**
- 7: AsyncLoad($T_{i,r}^{(d)}, \text{streamset}_{i,r,\text{load}}^{(d)}$)
- 8: $\text{eventset}_{i,r,\text{load}}^{(d)} \leftarrow \text{RecordEvents}(\text{streamset}_{i,r,\text{load}}^{(d)})$
- 9: **end for**
- 10: **end if**
- 11: **end procedure**
- 12: **procedure** FORWARDPASS(i)
- 13: **if** $i \in \mathcal{P}_r$ **then**
- 14: **for** $d \in \{1, 2\}$ **do**
- 15: WaitEvents($\text{eventset}_{i,r,\text{load}}^{(d)}, \text{stream}_{r,\text{infer}}$) $\triangleright$ RAW
- 16: **end for**
- 17: **end if**
- 18: LaunchKernels($i, \text{stream}_{r,\text{infer}}$)
- 19: $\text{event}_{i,r}^{\text{infer}} \leftarrow \text{RecordEvent}(\text{stream}_{r,\text{infer}})$
- 20: **if** $i \in \mathcal{P}_r$ **then**
- 21: $\text{last}_{b_i,r} \leftarrow i$ $\triangleright$ Mark slot b_i as last used by layer i
- 22: **end if**
- 23: **end procedure**

under the planner's residency budget (§4.3), reclaiming GPU memory for the KV cache while reducing exposed latency.

Memory hierarchy model. For a pageable layer on rank r , let $x_{\text{cpu},r}$ denote the fraction of its rank-local BFloat16 weight volume assigned to host DRAM; the remainder uses compressed GPU memory. The backends materialize their shares concurrently at effective bandwidths $\beta_{\text{cpu},r}$ and $\beta_{\text{gpu},r}$. As layers are homogeneous (§3), equalizing these backend times yields the same ideal CPU share for every layer:

$$x_{\text{cpu},r}^{\text{bw}} = \frac{\beta_{\text{cpu},r}}{\beta_{\text{gpu},r} + \beta_{\text{cpu},r}}. \quad (7)$$

This continuous target must be realized with whole tensors. Each layer contains L_r tensors of each type, and $\sigma_r^{(1)} = 2\sigma_r^{(2)}$. Thus, down-projection tensors comprise 1/3 of its total BFloat16 weight volume. In our deployments, $x_{\text{cpu},r}^{\text{bw}}$ is well below 1/3, so down-projection tensors alone can approximate the target. For each layer, the target CPU volume corresponds to $3L_r x_{\text{cpu},r}^{\text{bw}}$ down-projection tensors. FluxMoE rounds this count upward and assigns $\lceil 3L_r x_{\text{cpu},r}^{\text{bw}} \rceil$ such tensors to the CPU backend. Since rounding adds less than one

tensor, the resulting share $x_{\text{cpu},i,r}$ satisfies

$$0 \leq x_{\text{cpu},i,r} - x_{\text{cpu},r}^{\text{bw}} < \frac{1}{3L_r}. \quad (8)$$

Excluding budget-forced offloading, each layer’s rounded CPU allocation is within one down-projection tensor of the continuous target; fused tensors would double this bound.

FluxMoE derives the backend shares from the planner’s budget and the backends’ relative bandwidths, rather than scaling GPU allocation directly with model size. Thus, increasing an MoE model’s routed-expert parameter count does not require an equal increase in GPU-resident weight storage. GPU memory for expert weights grows with additional weights retained uncompressed or compressed on the GPU and any enlargement of two reusable blocks needed to hold rank-local pageable layers; offloaded weights reside in host DRAM.

Compressed GPU backend. FluxMoE uses compressed GPU memory as its primary expert tier, reducing the persistent footprint via on-demand decompression. Materialization is limited by the slower of local-memory reads and decompression. Modern GPUs’ local-memory bandwidth, nearly 1 TB/s to several TB/s, exceeds our measured decompression throughput of approximately 400 GB/s per GPU (§6.3); decompression determines the materialization rate.

FluxMoE compresses only routed-expert weights, leaving non-expert weights (e.g., attention projections) and shared-expert weights, if present, uncompressed and GPU-resident. Expert weights account for over 95% of the evaluated models’ parameter volume [20, 34, 40]. Compressing only routed experts therefore captures most memory savings while avoiding additional decompression work during the forward pass.

Our profiling finds BFloat16 expert weights exhibit the same entropy pattern as dense-model weights: low-entropy exponent bits and nearly uniform mantissa bits (Appendix A). FluxMoE therefore Huffman-encodes only the exponent bits offline and leaves the sign and mantissa bits unchanged, reducing routed-expert memory usage by 32.1–32.5% across the evaluated BFloat16 models [20, 34, 40] (§6.3). ZipMoE [41] also losslessly compresses BFloat16 exponent bits, but targets small-batch inference on unified-memory edge devices using LZ4HC or ZSTD with CPU decompression.

CPU offload backend. The compressed GPU backend provides high bandwidth but is still bounded by physical GPU memory capacity. To further reduce the GPU-resident parameter footprint, FluxMoE stores expert weights in pinned host memory and transfers them over PCIe via asynchronous DMA, following prior parameter-offloading systems [29, 32].

Modern accelerators provide tens of GB/s of PCIe bandwidth. After layer $i-1$ completes, FluxMoE materializes layer $i+1$ during layer i ’s execution, overlapping PCIe transfer and on-device decompression with inference. The two-layer window (§3) bounds in-flight data. PCIe alone often cannot supply expert weights fast enough to keep high-end GPUs

busy. Thus, the CPU backend acts as a complementary capacity tier, offloading weights to expand addressable memory.

4.3 Budget-Aware Residency Planner

On each GPU rank r , expert weights and KV-cache blocks compete for a fixed local memory budget. More expert residency reduces materialization cost but leaves less KV capacity; less residency does the reverse. FluxMoE’s budget-aware residency planner uses shared scheduler state and rank-local forward and materialization measurements as empirical control signals to select that rank’s residency adjustments.

Runtime control heuristic. Let the *remaining batch size* B_{rem} denote the number of unfinished sequences, including active sequences and those deferred or preempted by the scheduler. Both B_{rem} and B_{act} are shared across ranks and read from scheduler metadata in every decoding iteration.

On rank r , let $\tau_{\text{fwd},r}$ denote the forward-pass time measured before a residency adjustment. The planner profiles it with rank-local CUDA events, so it excludes materialization added by the candidate adjustment. We define rank r ’s *batch-normalized effective latency* as

$$\tau_{\text{eff},r} = \frac{B_{\text{rem}}}{B_{\text{act}}} \tau_{\text{fwd},r}. \quad (9)$$

Here, $\tau_{\text{eff},r}$ estimates the time for all B_{rem} sequences to advance by one token. The planner profiles materialization at layer granularity. It measures $\tau_{\text{load_layer}}^{(i,r)}$ with rank-local CUDA events, from load issuance until all participating local backends complete, and sets it to zero when layer i requires no materialization. Since successive pageable layers share the same PCIe-transfer and GPU-decompression pipelines, their loading times accumulate over a decoding iteration:

$$\tau_{\text{load},r} = \sum_{i=1}^N \tau_{\text{load_layer}}^{(i,r)}. \quad (10)$$

$\tau_{\text{load},r}$ sums per-layer materialization latencies and serves as an empirical control signal, not a measure of stall time. It includes latency hidden by inference; only materialization beyond a layer’s overlap window stalls its MoE block.

We combine KV-related batch pressure and aggregate materialization demand into an empirical control signal. We define the *inference-to-load ratio* as

$$\rho_r = \frac{\tau_{\text{eff},r}}{\tau_{\text{load},r}}. \quad (11)$$

The planner uses ρ_r together with B_{act} and B_{rem} to choose a residency adjustment. It evaluates ρ_r only during decoding; it disables residency adaptation during prefill, whose token parallelism hides most materialization.

Residency control model. Residency reclamation proceeds in two stages. The planner first converts all uncompressed-resident layers to pageable layers, keeping their tensors in compressed GPU memory. Once all layers are pageable, it enables tensor-level offloading and applies the hierarchy in §4.2 to partition pageable tensors between the two backends.

Restoration follows the reverse order. Layer-level conversion matches the materialization pipeline, while tensor-level offload limits additional PCIe traffic. The rules are:

- If $\rho_r > 1$ and $B_{\text{act}} < B_{\text{rem}}$, it performs the next reclamation.
- If $\rho_r < \theta$ (default: 0.9), it reverses one reclamation step.
- Otherwise, residency remains unchanged.

The first rule tests reclamation only when some unfinished sequences are not active and $\tau_{\text{load},r} < \tau_{\text{eff},r}$. This is an empirical switching condition rather than a guarantee of higher throughput. With a resizable KV allocator, reclaimed memory can expand rank r 's KV-cache allocation and reduce the batch-pressure factor $B_{\text{rem}}/B_{\text{act}}$. We set $\theta = 0.9$, making $[\theta, 1]$ a 10% dead zone that prevents small timing fluctuations from repeatedly reversing residency.

Runtime adaptation. The planner runs only at decoding control intervals with $B_{\text{act}} > 0$, ensuring that $\tau_{\text{eff},r}$ is defined. It skips intervals with no active sequences and terminates when $B_{\text{rem}} = 0$ (Algorithm 2). The profiling (line 3) follows a producer-consumer model. Inference and loading streams record CUDA timestamps asynchronously, while the planner consumes only complete measurements. Incomplete measurements are never used, and their events are reclaimed after completion without blocking producers or adding synchronization dependencies.

The *control interval* (CI) is the number of decoding iterations between planner runs. Within an engine launch, each run may update only rank-local pageable-tensor backend metadata; the update takes effect at the next decoding iteration. Layer-level residency changes require PagedTensor remapping and are applied only between engine launches (§6.4). For tensor-level control, the *adjustment granularity* (AG) defines a step relative to one layer's L_r rank-local weight tensors. Following the fine-grained placement in §4.2, each reclamation offloads up to $\lceil \text{AG} L_r \rceil$ remaining GPU-backed down-projection tensors. Once these are exhausted, it offloads up to $\lceil \text{AG} L_r / 2 \rceil$ fused gate/up-projection tensors to preserve approximately the same BFloat16 volume. Selection favors the layer with the smallest CPU-assigned volume, with ties broken by descending layer and rank-local expert indices. Restoration reverses the offload sequence. The released GPU memory is the sum of their compressed-copy sizes. Once no compressed GPU copies remain, no further expert memory can be reclaimed. We use $\text{CI}=16$ to balance response time and adjustment frequency and $\text{AG}=0.25$ to limit the loading increase per step.

5 Implementation

We prototype FluxMoE as middleware atop vLLM v0.23.0 [22]. The implementation comprises approximately 3.1 K LoC of C++/CUDA for PagedTensor's VMM allocator, stream/event coordination, and decompression kernels, and 2.1 K LoC of Python for expert registration, backend orchestration, residency planning, and vLLM integration.

Algorithm 2 Budget-Aware Residency Planner

Input: GPU rank r , threshold θ (default: 0.9), and initial rank-local residency state

- 1: **for** each decoding control interval with $B_{\text{act}} > 0$ **do**
- 2: Read shared B_{rem} and B_{act}
- 3: Read completed profiles $\tau_{\text{fwd},r}$ and $\tau_{\text{load_layer}}^{(i,r)}$
- 4: $\tau_{\text{load},r} \leftarrow \sum_{i=1}^N \tau_{\text{load_layer}}^{(i,r)}$
- 5: $\tau_{\text{eff},r} \leftarrow (B_{\text{rem}}/B_{\text{act}})\tau_{\text{fwd},r}$
- 6: $\rho_r \leftarrow \tau_{\text{eff},r}/\tau_{\text{load},r}$ $\triangleright +\infty$ for zero load
- 7: **if** $\rho_r > 1 \wedge B_{\text{act}} < B_{\text{rem}}$ **then**
- 8: Select the next rank-local reclamation step
- 9: **else if** $\rho_r < \theta$ **then**
- 10: Select one rank-local restoration step
- 11: **else**
- 12: Keep rank r 's residency unchanged
- 13: **end if**
- 14: **end for**

vLLM integration. We change 20 Python LoC across several vLLM functions: `create_weights` for allocating weight tensors on the CPU to avoid their full GPU footprint, `model-specific load_weights` for recording weight tensors' metadata, `load_model` for replacing vLLM's weight tensor storage with PagedTensor-managed tensors at stable virtual addresses, and `forward_native` for hooking materialization into MoE execution. The runtime hook waits for current-layer materialization before invoking the unchanged MoE kernel and triggers next-layer materialization afterwards.

PagedTensor. We implement PagedTensor using the CUDA VMM APIs `cuMemMap` and `cuMemUnmap` [4]. For each tensor type on rank r , it maintains two reusable physical blocks for pageable layers and dedicated blocks for uncompressed-resident layers (§6.4).

Expert memory backends. We implement the compressed GPU and CPU offload backends. The GPU backend selectively Huffman-encodes expert exponent bits [17] and reconstructs them on demand with high-throughput decompression kernels [46]; the CPU backend asynchronously transfers weights from pinned host DRAM over PCIe. For each layer and rank, PagedTensor assigns each weight-tensor type one decompression and one PCIe-transfer stream, totaling four concurrent loading streams. Under EP, FluxMoE uses vLLM's expert map to identify rank-local expert weights without changing routing or communication.

Budget-aware residency planner. The planner configures uncompressed GPU residency per layer and compressed-GPU or host-DRAM backing per tensor. Within an engine launch, only pageable-tensor backing can change, which takes effect at decoding-iteration boundaries. All prefetches in iteration k use immutable metadata version ver_k . For each retired compressed GPU copy, every loading stream records a CUDA event after its final operation that reads the copy. Af-

Table 2. Evaluated models and GPU memory budgets.

Model	MoE Layers/Routed Experts per Layer	GPUs	Budget – Weights = Headroom (GB)
GLM-4.5 [34]	89/160	8×H20	738.7 – 710 = 28.7
Qwen3-Next [40]	48/512	4×L40S	173.9 – 160 = 13.9
Mixtral [20]	32/8	2×L40S	86.9 – 94 = -7.1

ter issuing all iteration- k prefetches, the planner atomically publishes ver_{k+1} before any iteration- $(k + 1)$ prefetch, ensuring subsequent prefetches use the new backend. The old copy remains allocated until all recorded events complete; FluxMoE then releases it without device-wide synchronization. Within a launch, memory released by tensor offloading returns to the GPU allocator but cannot expand vLLM’s KV pool, whose capacity is fixed at initialization. Our integration fixes layer residency within each launch, so joint expert-KV budget changes are applied only between launches (§6.4).

6 Evaluation

6.1 Experimental Setup

Testbed. We use two servers with different resource profiles. The L40S server has two 24-core Intel Xeon Gold 5318Y CPUs, 2 TB of host DRAM, and 4 NVIDIA L40S GPUs, each with 48 GB nominal memory (48.3 GB reported by `nvidia-smi`), connected via PCIe Gen4 with 32 GB/s peak per-GPU H2D bandwidth. The H20 server has two 48-core Intel Xeon Platinum 8468 CPUs, 2 TB of host DRAM, and 8 NVIDIA H20 GPUs, each with 96 GB nominal memory (102.6 GB reported by `nvidia-smi`), connected via PCIe Gen5 with 64 GB/s peak per-GPU H2D bandwidth.

Models. We evaluate FluxMoE on three MoE models. GLM-4.5 [34] is a production-scale MoE whose weights exceed the aggregate memory of eight 80-GB H100 GPUs; Mixtral-8×7B-Instruct [20] (Mixtral for short) is widely studied; and Qwen3-Next-80B-A3B-Instruct [40] (Qwen3-Next for short) falls between them in size. Table 2 summarizes the model architectures, deployments, and memory footprints. All deployments use vLLM’s default 90% GPU-memory budget [22]. Unlike §2’s 4×L40S setup for isolating KV-capacity effects, we evaluate Mixtral on a constrained 2×L40S deployment, where its weights exceed vLLM’s 90% GPU-memory budget.

Dataset and workloads. We sample input prompts from ShareGPT [3], following prior LLM serving evaluations [19, 38, 43, 51]. Mean tokenized lengths are 44.9, 45.6, and 52.1 tokens for GLM-4.5, Qwen3-Next, and Mixtral, respectively, with a maximum of 1,121. Except for online-serving evaluation, all experiments use offline serving, with all requests available initially. We sweep either (i) $B_{\text{cfg}} \in \{32, 64, 128, 256\}$ with output length fixed at 4,096 or (ii) output length in $\{1,024, 2,048, 3,072, 4,096\}$ tokens with $B_{\text{cfg}} = 256$. We report the average results over five runs, with error bars as 95% confidence intervals under the Student’s t -distribution.

Metrics. We report output throughput (tokens/s), average

Time-To-First-Token (TTFT), and average Time-Per-Output-Token (TPOT) for offline serving, and End-to-End (E2E) latency for online serving. TTFT, TPOT, and E2E measure submission to the first token, the mean interval between subsequent tokens, and submission to completion, respectively.

System configurations. All systems use a 512 GB aggregate host-DRAM tier for KV cache offloading, large enough to hold the full KV cache of any evaluated workload. For a fair comparison, systems that offload expert weights to host DRAM use a common budget of 12.5% of routed-expert weights, the minimum needed for Mixtral to fit in baselines. We configure four baselines and FluxMoE as follows:

- **vLLM [22]:** A standard weight-resident serving system that keeps all expert weights in GPU memory.
- **vLLM-O:** A vLLM variant that tests whether whole-layer CPU weight offloading with prefetching suffices. It keeps earlier MoE layers uncompressed on the GPU, offloads the final layers covered by the budget to host DRAM, and prefetches layer $i + 1$ while layer i executes.
- **FluxMoE-H:** An ablation of the bandwidth-balanced hierarchy (§4.2) that places each layer entirely in one backend, using compressed GPU memory for earlier layers and host DRAM for the final layers covered by the budget.
- **KTransformers [7]:** A hybrid MoE system that distributes expert weights and computation across GPUs and CPUs. Rather than offloading entire layers, it distributes offloaded experts across all MoE layers. It uses TP only as it lacks EP support [2], while all other systems use both TP and EP. Following its configuration guide [1], we use 48 (96) inference threads on the L40S (H20) server, one per physical core, and two thread pools, one per NUMA node.
- **FluxMoE:** For GLM-4.5 and Qwen3-Next, it keeps all routed experts in compressed GPU memory. As weight-resident vLLM fits on both deployments, this isolates compression’s benefits and overheads. For Mixtral, we use a fixed placement to match the aggregate offloading budget, independently of the down-projection-first policy in §4.3. Per layer, we offload one fused gate/up-projection tensor from one GPU and one down-projection tensor from the other (64 tensors total). For a fair comparison with KTransformers, we evaluate FluxMoE-TP, a TP-only variant retaining FluxMoE’s mechanisms. It performs comparably to FluxMoE in offline and online experiments (Appendix B).

6.2 Overall Performance

(Exp#1) Memory-sufficient regime. Figure 4 shows that FluxMoE’s advantage grows with KV-cache pressure. For GLM-4.5, at $B_{\text{cfg}} = 256$, FluxMoE delivers 7.2×, 2.6×, and 3.5× the throughput of vLLM, vLLM-O, and KTransformers, respectively. vLLM peaks at $B_{\text{cfg}} = 64$ and then declines as limited KV capacity reduces B_{act} . vLLM-O supports a larger B_{act} than vLLM through offloading but trails FluxMoE since expert prefetching competes with KV-cache swapping over PCIe. At large batches, nearly every expert receives tokens,

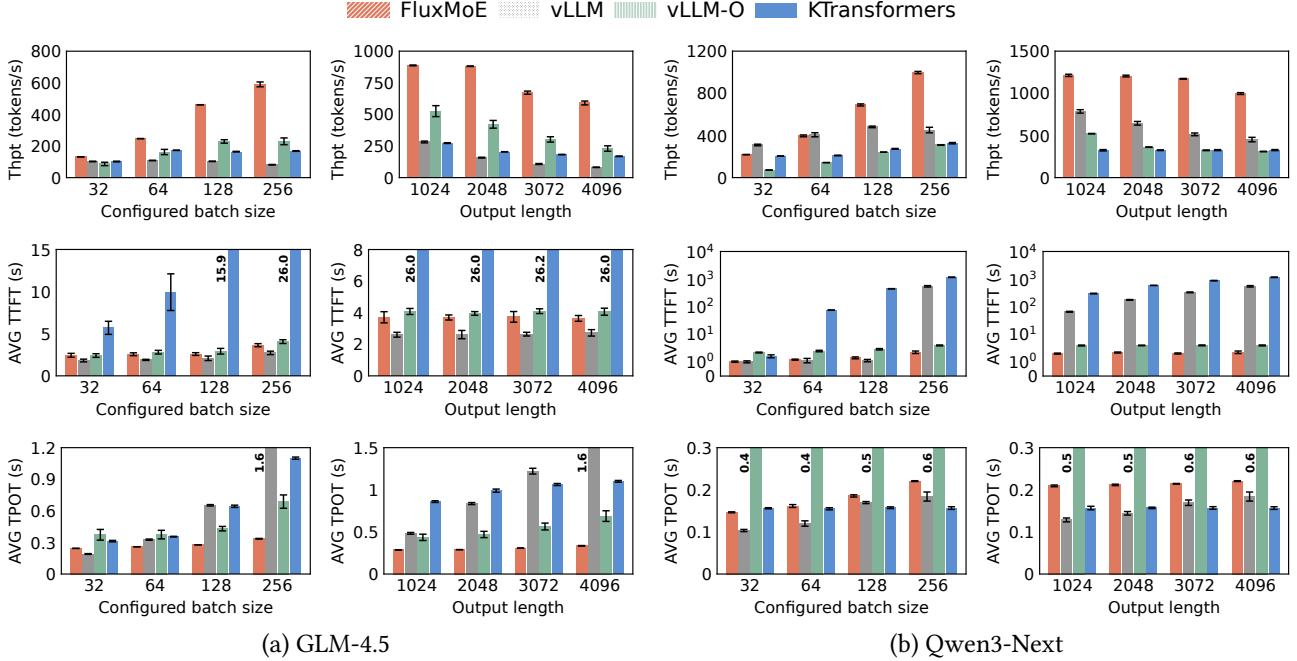


Figure 4. (Exp#1) Memory-sufficient regime. For each model, left: output length = 4,096; right: $B_{\text{cfg}} = 256$.

so CPU expert computation and output merging become KTransformers’s throughput bottlenecks.

Qwen3-Next exposes the compression trade-off. At $B_{\text{cfg}} = 32$, FluxMoE achieves 70.5% of vLLM’s throughput since the saved memory does not raise B_{act} ; the resulting unamortized decompression cost is expected. At $B_{\text{cfg}} = 256$ and an output length of 4,096 tokens, additional KV capacity raises B_{act} , giving FluxMoE 2.2× vLLM’s throughput.

FluxMoE also sustains throughput better as output length grows. From 1,024 to 4,096 tokens, its throughput drops by 33.5% on GLM-4.5 and 17.8% on Qwen3-Next, versus 71.1% and 42.5% for vLLM. Its smaller expert footprint preserves KV capacity and a higher B_{act} as tokens accumulate.

We next examine latencies at $B_{\text{cfg}} = 256$ and an output length of 4,096. On GLM-4.5, FluxMoE reduces TTFT and TPOT by 10.4% and 51.3% versus vLLM-O, and by 86.0% and 69.6% versus KTransformers, respectively, by avoiding vLLM-O’s CPU-to-GPU transfers and KTransformers’s CPU expert execution and output merging. Compared with vLLM, FluxMoE has 33.6% higher TTFT but 79.0% lower TPOT on GLM-4.5, and 99.6% lower TTFT but 19.9% higher TPOT on Qwen3-Next. With limited KV capacity, vLLM preempts running GLM-4.5 requests, raising TPOT, but delays Qwen3-Next admission, raising TTFT.

(Exp#2) Memory-constrained regime. On Mixtral, we evaluate expert-placement strategies under the same offload budget (Figure 5). At $B_{\text{cfg}} = 256$ and an output length of 4,096, FluxMoE delivers 8.7× and 4.3× the throughput of vLLM-O and KTransformers, respectively. It reduces TTFT and TPOT by 19.8% and 86.9% versus vLLM-O, and by 99.9% and 29.1% versus KTransformers. Both baselines keep GPU-resident

experts uncompressed, providing less KV capacity and a lower B_{act} . In addition, vLLM-O’s whole-layer prefetching contends with KV-cache swapping over PCIe, while KTransformers is limited by CPU expert execution and output merging. FluxMoE increases KV capacity and B_{act} through compression and reduces PCIe contention by distributing offloaded weights across layers. Its larger B_{act} raises throughput, whereas KTransformers’s limited KV capacity delays prefill, concentrating the latency gap in TTFT.

At $B_{\text{cfg}} = 256$ and an output length of 4,096, FluxMoE improves FluxMoE-H’s throughput by 25.3% and reduces its TTFT and TPOT by 28.0% and 21.1%, respectively. Their equal compressed and offloaded parameter volumes provide the same KV capacity, so the difference comes from materialization: FluxMoE uses both backends per layer, whereas FluxMoE-H’s CPU-backed layers rely entirely on PCIe and stall longer.

6.3 Expert Materialization Analysis

(Exp#3) Lossless decompression benchmark. This experiment validates the compressed GPU backend by checking whether it saves GPU memory and decompresses fast enough to avoid inference stalls. For each model, we benchmark FluxMoE’s selective Huffman decoder (§4.2) on one GPU of its deployment type (Table 2). For a model with N MoE layers on G GPUs, each of the $N \times G$ runs decodes the routed-expert shard assigned to one layer on one GPU. Each expert’s two projection-weight tensors are decoded concurrently on separate CUDA streams, matching inference. We report mean $\pm$ standard deviation for latency and throughput, defined as uncompressed BFloat16 volume divided by decoding time.

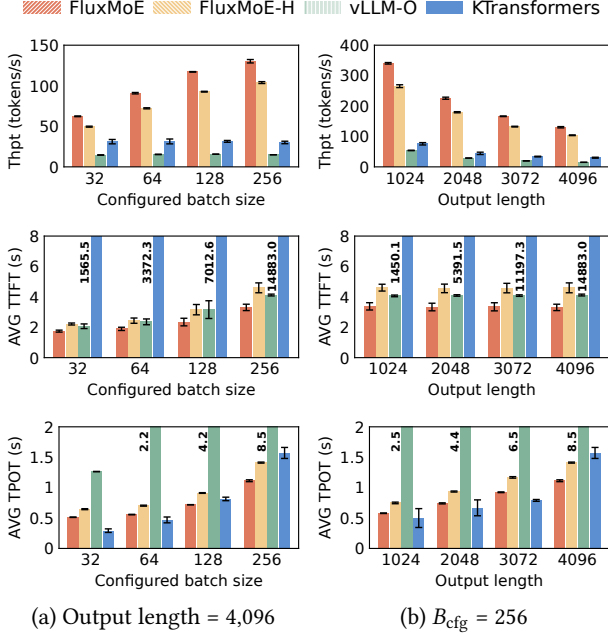


Figure 5. (Exp#2) Memory-constrained regime for Mixtral.

Table 3 shows 32.1–32.5% lossless storage savings; Appendix C confirms unchanged vLLM perplexity, showing that compression does not affect model quality. Decompression reaches 396.5–435.4 GB/s, or 6.8–12.6× the corresponding peak PCIe H2D bandwidth, supporting the placement of most pageable parameter volume in the compressed GPU backend (§4.2). For GLM-4.5 and Mixtral, the 2.17–3.07 ms latency fits within Exp#4’s 5.16–21.50 ms stall-free windows, so forwarding can hide decompression.

(Exp#4) Per-layer expert-materialization breakdown.

We measure how much materialization overlaps forwarding and whether bandwidth balancing reduces exposed stalls under inference contention. GLM-4.5 covers decompression-only materialization, while Mixtral covers concurrent PCIe loading and GPU decompression, together representing both paths in §4.2. CUDA events on the loading and inference streams measure per-layer E2E materialization and the corresponding stall-free window; any excess is reported as average stall across layers. We exclude PagedTensor mapping since it occurs only during initialization (§4.1).

Table 4 shows that FluxMoE keeps expert materialization largely hidden behind forwarding. For GLM-4.5, materialization takes 2.17 ms within a 5.16–5.82 ms stall-free window, exposing less than 0.01 ms of average stall per layer. For Mixtral at $B_{\text{cfg}} = 256$, average stall is 33.9% of FluxMoE’s stall-free window, versus 51.2% for FluxMoE-H after averaging its CPU- and GPU-backed layer windows, and FluxMoE reduces average per-layer stall duration by 43.3% compared with FluxMoE-H. The reduction comes from materializing each layer through both backends concurrently (§4.2), whereas FluxMoE-H’s CPU-backed layers depend entirely on PCIe.

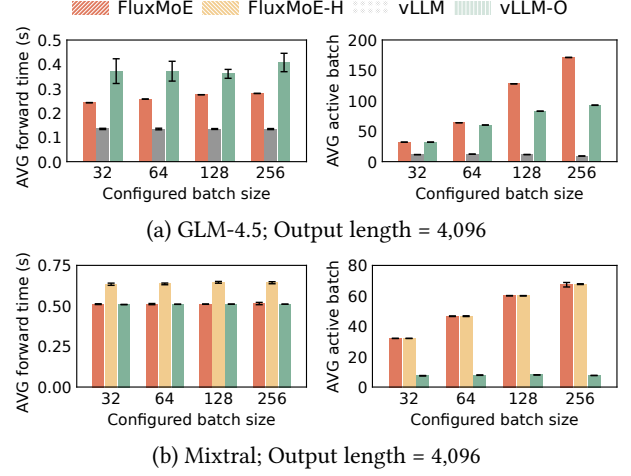


Figure 6. (Exp#4) Average forward time per decoding iteration (left) and average active batch size (right).

For FluxMoE, PCIe transfer and GPU decompression take 15.33 ms and 3.13 ms, respectively, while E2E materialization takes 27.93 ms since it also includes queueing, stream waits, and launch delay.

Expert materialization remains stable as batch size grows. From $B_{\text{cfg}} = 32$ to 256, GLM-4.5’s E2E materialization latency stays at 2.17 ms, while Mixtral’s changes by less than 1%, with no systematic increase in exposed stall. Intermediate results at $B_{\text{cfg}} = 64$ and 128 in Appendix B support this conclusion.

Figure 6 shows the cost-benefit trade-off behind FluxMoE’s throughput gain: each iteration takes longer but processes more sequences. For GLM-4.5 at $B_{\text{cfg}} = 256$, compared with vLLM, FluxMoE increases $\bar{B}_{\text{act}}$ by 17.6× (benefit) and forward time by 1.1× (cost), due to the larger batch and GPU contention from decompression. For Mixtral, compared with vLLM-O, FluxMoE increases $\bar{B}_{\text{act}}$ by 7.8× (benefit) while increasing forward time by only 0.8% (cost). Compared with FluxMoE-H, which has the same expert footprint, KV capacity, and nearly identical $\bar{B}_{\text{act}}$, FluxMoE’s lower materialization stall reduces forward time by 20.0%. This analysis is consistent with Figures 4 and 5.

6.4 Residency Planner Analysis

The planner adjusts expert placement between GPU and CPU in response to KV-cache pressure (§4.3). We evaluate sensitivity to the planner’s control interval (CI) and adjustment granularity (AG) (Exp#5), then compare adaptive and fixed residency under changing KV-cache pressure (Exp#6). **(Exp#5) Residency planner sensitivity.** We compare against fixed residency with all routed-expert tensors compressed in GPU memory. At an output length of 4,096, we use $B_{\text{cfg}} = 256$ for GLM-4.5 and Qwen3-Next, but $B_{\text{cfg}} = 32$ for Mixtral since its no-offload configuration leaves insufficient GPU memory for runtime state at $B_{\text{cfg}} = 256$. Before the first adjustment, the default CI=16 corresponds to 3.0–4.5 s across

Table 3. (Exp#3) Lossless decompression benchmark (mean $\pm$ std.). Uncompressed size is the BFloat16 volume decoded per layer per GPU; for Mixtral, it covers the 87.5% assigned to the compressed GPU backend.

Model	GPU	Uncompressed size (GB)	Storage saved (%)	AVG latency (ms)	P99 latency (ms)	Thpt (GB/s)
Mixtral	L40S	1.23	32.4	3.07 ± 0.14	3.13 ± 0.14	401.9 ± 0.8
Qwen3-Next	L40S	0.81	32.1	2.03 ± 0.01	2.07 ± 0.02	396.5 ± 2.3
GLM-4.5	H20	0.94	32.5	2.17 ± 0.01	2.17 ± 0.01	435.4 ± 2.6

Table 4. (Exp#4) Per-layer expert materialization at output length 4,096 (mean $\pm$ std., ms). CPU Offloading and Decompression report isolated PCIe-transfer and decompression time, respectively. E2E materialization spans load issuance to tensor readiness, including queueing, stream waits, and launch delay; Stall-free window spans issuance until tensor use; AVG stall is the remaining wait, averaged across layers. For FluxMoE-H, C and D denote CPU- and compressed-GPU-backed layers, respectively.

Model	Method	B_{cfg}	CPU Offloading	Decompression	E2E materialization	Stall-free window	AVG stall
GLM-4.5	FluxMoE	32	–	2.17 ± 0.01	2.17 ± 0.01	5.16 ± 0.43	< 0.01
	FluxMoE	256	–	2.17 ± 0.01	2.17 ± 0.01	5.82 ± 1.15	< 0.01
Mixtral	FluxMoE	32	15.37 ± 5.00	3.12 ± 0.15	28.13 ± 1.62	21.19 ± 6.66	7.72 ± 4.97
	FluxMoE-H	32	126.10 ± 12.54	3.50 ± 0.01	C: 200.74 ± 33.34 D: 3.51 ± 0.01	C: 100.30 ± 64.41 D: 13.46 ± 19.00	13.05 ± 36.27
	FluxMoE	256	15.33 ± 4.95	3.13 ± 0.15	27.93 ± 1.53	21.50 ± 6.65	7.29 ± 4.79
	FluxMoE-H	256	140.58 ± 12.12	3.51 ± 0.01	C: 204.00 ± 29.46 D: 3.52 ± 0.02	C: 105.27 ± 70.27 D: 13.66 ± 16.37	12.86 ± 37.18

models. We sweep CI over $\{8, 16, 32, 64\}$ at AG=0.25 and AG over $\{0.25, 0.5, 0.75, 1.0\}$ at CI=16. All runs remain in the down-projection stage: each reclamation offloads $[AGL_r]$ tensors per rank across pageable layers (L_r at AG=1.0); no fused gate/up-projection tensor is selected.

Figure 7 shows that the default CI=16 and AG=0.25 remain close to fixed residency while releasing 48.8 GB, 6.9 GB, and 1.9 GB of GPU memory for GLM-4.5, Qwen3-Next, and Mixtral, respectively, equal to 10.8%, 6.6%, and 3.1% of their compressed routed-expert storage (Appendix B). H20’s higher PCIe bandwidth permits more adjustments for GLM-4.5, while Mixtral’s larger tensors quickly make loading limiting, explaining its smaller release and lower sensitivity to CI and AG. For GLM-4.5 and Qwen3-Next, all settings initially track fixed residency while $B_{\text{act}} = B_{\text{cfg}}$. Once offloading begins, CI values of 16, 32, and 64 remain close to the baseline, whereas CI=8 releases 2.0 $\times$ and 1.9 $\times$ as much GPU memory as CI=16, increasing PCIe traffic and reducing throughput. At CI=16, larger AG lowers throughput by offloading more tensors per adjustment; AG=0.25 remains closest to the baseline.

(Exp#6) Emulated runtime residency adaptation. As vLLM fixes KV capacity at initialization, we emulate adaptation by splitting a workload with $B_{\text{cfg}} = 256$ and output length 4,096 across 41 engine launches. The first receives the original prompts; later launches concatenate each prompt with its accumulated output to reconstruct the growing KV state. Each of the first 40 produces at least 256×100 new tokens in aggregate; the final launch produces the remainder.

The first launch maximizes expert residency after reserving other runtime state and enough KV cache to prefill one request. Before each subsequent launch, the planner uses the

preceding launch’s measurements to adjust residency: it first makes uncompressed layers pageable, potentially several at once since each launch spans multiple control intervals, and then offloads tensors with AG=0.25 once all layers are pageable. These transitions reverse as KV pressure falls.

We concatenate decoding iterations across launches. To summarize performance, we compute *aggregate throughput* as the total output tokens divided by the sum of the initial launch’s prefill time and all decoding times, excluding reconstruction prefills and relaunch overhead. The baseline uses FluxMoE’s configuration from §6.1 in one launch; replaying the configuration across 41 launches changes aggregate throughput by less than 5% across models.

Figure 8 shows that the planner initially achieves higher per-iteration throughput than fixed residency on all three models by keeping more expert weights uncompressed and GPU-resident. As KV pressure grows, paging and offloading expand KV capacity at the cost of loading overhead; the final throughput drop reflects request completion. For GLM-4.5 and Qwen3-Next, reclaimed memory reduces request suspension and sustains a larger $\bar{B}_{\text{act}}$, allowing completion in fewer iterations. For Mixtral, the planner requires more iterations and has a smaller $\bar{B}_{\text{act}}$ than fixed residency, but maintains higher per-iteration throughput through most of decoding. Under this multi-launch emulation, adaptive residency improves aggregate throughput over fixed residency by 20.9%, 16.9%, and 42.7% for GLM-4.5, Qwen3-Next, and Mixtral, respectively. Because a dynamic-KV runtime would incur unmodeled in-run resizing and reconfiguration overhead, these gains are an optimistic estimate of the potential benefit of integrating a resizable KV allocator.

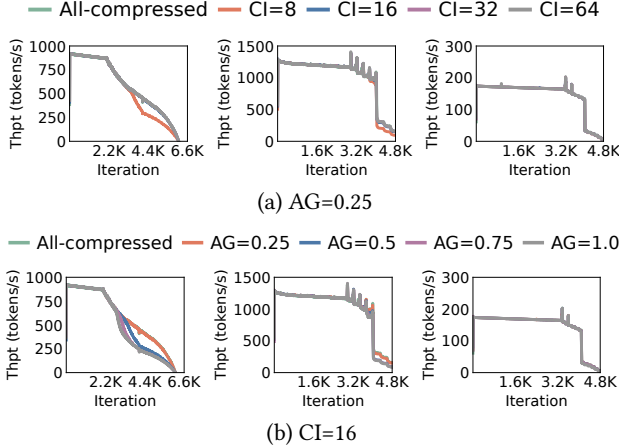


Figure 7. (Exp#5) Residency planner sensitivity. From left to right: GLM-4.5, Qwen3-Next, and Mixtral.



Figure 8. (Exp#6) Emulated runtime residency adaptation. From left to right: GLM-4.5, Qwen3-Next, and Mixtral.

6.5 Online Serving Performance

We next test whether FluxMoE also reduces latency under asynchronous arrivals and varying online load.

(Exp#7) Online serving performance. We use the same ShareGPT prompts [3] as in offline evaluation, but keep their original response lengths. Following prior evaluations [13, 22, 44, 51], we model request arrivals as a Poisson process and evaluate FluxMoE at 5, 10, 20, and 40 req/s, reporting 10 req/s here and the others in Appendix B. Each run uses 10 warm-up and 1,000 measured requests. We set vLLM’s concurrency limit, `max_num_seqs`, to 256 to match the largest offline B_{cfg} , or 512 to increase KV pressure. The realized B_{act} is bounded by this limit, the batch supported by KV capacity, and the number of ready requests. At 10 req/s, all systems saturate at limit 256; at 512, only FluxMoE on GLM-4.5 and Qwen3-Next remains unsaturated.

Figure 9 shows the E2E-latency CDFs. Across all three models, FluxMoE outperforms KTransformers because it avoids CPU expert computation and result merging. For GLM-4.5 at a concurrency limit of 256, FluxMoE’s P50 is 4.6% above vLLM’s since vLLM’s resident weights incur no materialization. Raising the concurrency limit from 256 to 512 allows FluxMoE to exploit its larger KV capacity and sustain a larger B_{act} than vLLM. The resulting batching benefit outweighs materialization, lowering P50 by 67.3% versus vLLM and 24.9% versus vLLM-O. The latter frees KV memory but still incurs whole-layer PCIe prefetching. FluxMoE avoids manually switching between vLLM’s weight-resident mode

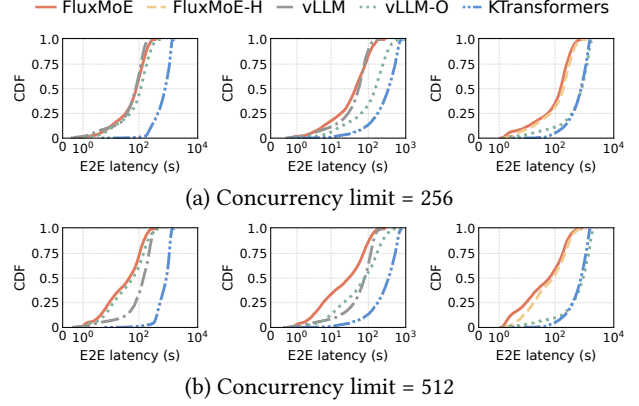


Figure 9. (Exp#7) Online serving performance at 10 req/s. From left to right: GLM-4.5, Qwen3-Next, and Mixtral.

and vLLM-O’s offloading mode as online conditions change.

For Mixtral, where vLLM cannot fit, FluxMoE lowers P50 by 18.7% (24.1%) over FluxMoE-H at limit 256 (512). Since both provide the same KV capacity, FluxMoE’s gain comes from bandwidth-balanced allocation, which reduces materialization overhead. Across 5, 20, and 40 req/s, FluxMoE has the lowest P50 in all three models with a limit of 512, matching the same trend observed at 10 req/s (Appendix B).

7 Related Work

MoE expert offloading. Earlier dense-model systems place parameters across GPU, CPU, and NVMe to run models beyond GPU capacity [29, 32]. In MoE models, loading experts from lower-tier memory creates massive PCIe traffic and makes bandwidth a throughput bottleneck [6, 21, 39]. MoE systems instead offload experts through four approaches: (i) CPU-GPU co-inference systems execute expert computation on both devices [7, 21, 49] or pipeline computation with weight transfers [6]; (ii) Prediction-based methods exploit request histories [39, 45], cross-layer routing signals [11], or learned predictors [14] to cache and prefetch experts; some further combine prediction with adaptive gating [48] or task-aware residency [33]; (iii) Model-level approaches modify routing to reveal future expert choices [18] or fine-tune it to concentrate expert demand into a smaller working set [30]; and (iv) Batched-inference methods improve cache reuse and overlap expert loading through priority-based expert caching [23] or expert-aware multi-batch scheduling [10]. FluxMoE keeps expert computation on GPUs and preserves routing, avoiding CPU expert-compute, output-merge, and prediction-miss overheads. Using the fixed layer order, FluxMoE materializes next-layer experts from compressed GPU memory and host DRAM concurrently, reducing PCIe stalls while freeing GPU memory for the KV cache.

KV-cache management in LLMs. KV-cache management follows two lines of work. The first optimizes cache layout and reuse within GPU memory to reduce fragmentation and redundant storage [22, 42, 43, 47]; the second moves KV

state across devices and storage tiers to expand capacity or support disaggregated serving [25, 28, 50]. All assume model weights are persistent GPU-resident state; FluxMoE breaks this assumption by making experts dynamic and evictable, freeing memory for the KV cache these systems manage.

8 Conclusion

FluxMoE is an MoE serving system that decouples experts from persistent GPU residency via expert paging. It employs PagedTensor, a two-tier memory hierarchy, and a budget-aware residency planner. Experiments on NVIDIA GPUs show that FluxMoE achieves up to $7.2\times$ vLLM’s throughput and 79.0% lower TPOT for GLM-4.5, and $4.3\times$ the throughput of KTransformers for Mixtral-8 $\times$ 7B-Instruct, when vLLM cannot fit, without measurable model-quality loss.

References

- [1] [n.d.]. KT-Kernel configuration guide for KTransformers. <https://github.com/kvccache-ai/ktransformers/blob/main/kt-kernel/README.md#kt-kernel-parameters>.
- [2] [n.d.]. KTransformers GitHub repository. <https://github.com/kvccache-ai/ktransformers>.
- [3] [n.d.]. ShareGPT datasets. <https://huggingface.co/collections/bunnycore/sharegpt-datasets-66fa831dcee14c587f1e6d1c>.
- [4] [n.d.]. Virtual memory management APIs in CUDA programming guide. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/virtual-memory-management.html>.
- [5] Pranjal Aggarwal and Sean Welleck. 2025. L1: Controlling how long a reasoning model thinks with reinforcement learning. In *Second Conference on Language Modeling*. <https://openreview.net/forum?id=4jdlxXBNve>
- [6] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *Proc. of ACM ASPLOS*. doi:10.1145/3669940.3707267
- [7] Hongtao Chen, Weiyou Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proc. of ACM SOSP*. doi:10.1145/3731569.3764843
- [8] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Jun-Mei Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiaoling Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bing-Li Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dong-Li Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, JingChang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Jiong Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, M. Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, Ruiqi Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shao-Kang Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wen-Xia Yu, Wentao Zhang, Wangding Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyu Jin, Xi-Cheng Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yi Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yu-Jing Zou, Yujia He, Yunfan Xiong, Yu-Wei Luo, Yu mei You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yao Li, Yi Zheng, Yuchen Zhu, Yunxiang Ma, Ying Tang, Yukun Zha, Yuting Yan, Zehui Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhen guo Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zi-An Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645 (2025), 633 – 638. doi:10.1038/s41586-025-09422-z
- [9] Ruibo Fan, Xiangrui Yu, Xinglin Pan, Zeyu Li, Weile Luo, Qiang Wang, Wei Wang, and Xiaowen Chu. 2026. ZipServ: Fast and memory-efficient LLM inference with hardware-aware lossless compression. In *Proc. of ACM ASPLOS*. doi:10.1145/3779212.3790250
- [10] Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. Klotki: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline. In *Proc. of ACM ASPLOS*. doi:10.1145/3676641.3716261
- [11] Zhiyuan Fang, Xingfan Yu, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, and Fan Yu. 2026. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. In *Proc. of ACM WWW*. doi:10.1145/3774904.3792527
- [12] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations* (2023). <https://arxiv.org/abs/2210.17323>
- [13] Shiwei Gao, Qing Wang, Shaoxun Zeng, Youyou Lu, and Jiwei Shu. 2025. Weaver: Efficient multi-LLM serving with attention offloading. In *Proc. of USENIX ATC*. <https://www.usenix.org/conference/atc25/presentation/gao>
- [14] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. 2026. ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling. In *Proc. of ACM/IEEE DAC*. <https://arxiv.org/abs/2410.17954>
- [15] Anat Heilper and Doron Singer. 2025. Lossless compression of neural network components: Weights, checkpoints, and K/V caches in low-precision formats. *arXiv preprint arXiv:2508.19263* (2025). <https://arxiv.org/abs/2508.19263>
- [16] Moshik Herscovitch, Andrew Wood, Leshem Choshen, Guy Gironmsky, Roy Leibovitz, Or Ozeri, Ilias Ennmouri, Michal Malka, Peter Chin, Swaminathan Sundararaman, and Danny Harnik. 2025. ZipNN: Lossless compression for AI models. In *Proc. of IEEE CLOUD*. doi:10.1109/CLOUD67622.2025.00028
- [17] David A Huffman. 1952. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101. doi:10.1109/JRPROC.1952.273898
- [18] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proc. of ACM/IEEE ISCA*. doi:10.1109/ISCA59077.2024.00078
- [19] Jinwoo Jeong and Jeongseob Ahn. 2025. Accelerating LLM serving for multi-turn dialogues with efficient resource management. In *Proc. of ACM ASPLOS*. doi:10.1145/3676641.3716245
- [20] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Men-

- sch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024). <https://arxiv.org/abs/2401.04088>
- [21] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=N5fVv6PZGz>
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proc. of ACM SOSP*. doi:10.1145/3600006.3613165
- [23] Kexin Li, Wenkan Huang, Qinggang Wang, Long Zheng, Xiaofei Liao, Hai Jin, and Jingling Xue. 2025. Diff-MoE: Efficient batched MoE inference with priority-driven differential expert caching. In *Proc. of ACM SC*. doi:10.1145/3712285.3759903
- [24] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html
- [25] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proc. of ACM SIGCOMM*. doi:10.1145/3651890.3672274
- [26] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Byj72udxe>
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas K  pf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019), 8026 – 8037. <https://proceedings.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [28] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2026. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *ACM Trans. on Storage* (2026). doi:10.1145/3773772
- [29] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning. In *Proc. of ACM SC*. doi:10.1145/3458817.3476205
- [30] Arian Raje, Anupam Nayak, and Gauri Joshi. 2026. MELINOE: Fine-tuning enables memory-efficient inference for mixture-of-experts models. *arXiv preprint arXiv:2602.11192* (2026). <https://arxiv.org/abs/2602.11192>
- [31] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A flexible and efficient RLHF framework. In *Proc. of ACM EuroSys*. doi:10.1145/3689031.3696075
- [32] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher R  , Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*. <https://proceedings.mlr.press/v202/sheng23a.html>
- [33] Suraiya Tairin, Shohaib Mahmud, Haiying Shen, and Anand Iyer. 2025. eMoE: Task-aware memory efficient mixture-of-experts-based (MoE) model inference. *arXiv preprint arXiv:2503.06823* (2025). <https://arxiv.org/abs/2503.06823>
- [34] GLM-4.5 Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianshan Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, Mingshu Zhai, Pengfan Du, Qian Dong, Shangde Lei, Shangqing Tu, Shangting Yang, Shaoyou Lu, Shijie Li, Shuang Li, Shuang-Li, Shuxun Yang, Sibo Yi, Tianshu Yu, Wei Tian, Weihang Wang, Wenbo Yu, Weng Lam Tam, Wenjie Liang, Wentao Liu, Xiao Wang, Xiaohan Jia, Xiaotao Gu, Xiaoying Ling, Xin Wang, Xing Fan, Xingru Pan, Xinyuan Zhang, Xinze Zhang, Xiuqing Fu, Xunkai Zhang, Yabo Xu, Yandong Wu, Yida Lu, Yidong Wang, Yilin Zhou, Yiming Pan, Ying Zhang, Yingli Wang, Yingru Li, Yinpei Su, Yipeng Geng, Yitong Zhu, Yongkun Yang, Yuhang Li, Yuhao Wu, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yuxuan Zhang, Zezhen Liu, Zhen Yang, Zhengda Zhou, Zhongpei Qiao, Zhuoer Feng, Zhuorui Liu, Zichen Zhang, Zihan Wang, Zijun Yao, Zikang Wang, Ziqiang Liu, Ziwei Chai, Zixuan Li, Zuodong Zhao, Wenguang Chen, Jidong Zhai, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. 2025. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. *arXiv preprint arXiv:2508.06471* (2025). <https://arxiv.org/abs/2508.06471>
- [35] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jiale Cui, Haochen Ding, Mengxiao Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jia-Xing Guo, Hao-Xing Hu, Xiaoru Hao, Tian He, Weiran He, Wen He, Chao Hong, Yan-Ni Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hong-Li Lin, Xiaohan Lin, Zongyu Lin, Chengyi Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tian-Bo Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Li Lu, Shen Ma, Xinyu Ma, Yi-Xuan Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Li-Na Shi, Sheng-Rong Shi, Feifan Song, Jian-Fei Su, Zhen-Xin Su, Xinjie Sun, Flood Sung, Heyi Tang, Ji-Hua Tao, Qi Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qi-Feng

- Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiao-Ming Xie, Weiming Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Zi-Yang Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhengqi Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wen guang Ye, Zhuorui Ye, Bohong Yin, Long Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Hao-Dong Zhao, Yikai Zhao, Huabin Zheng, Shao Jian Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhengxin Zhu, Weiyu Zhuang, and Xinxing Zu. 2025. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534* (2025). <https://arxiv.org/abs/2507.20534>
- [36] Philippe Tillett, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. of ACM MAPL*. doi:10.1145/3315508.3329973
- [37] Yuxuan Tong, Xiwen Zhang, Rui Wang, Ruidong Wu, and Junxian He. 2024. DART-Math: Difficulty-aware rejection tuning for mathematical problem-solving. *Advances in Neural Information Processing Systems* 37 (2024), 7821–7846. doi:10.52202/079017-0251
- [38] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. 2025. Aegaeon: Effective GPU pooling for concurrent LLM Serving on the market. In *Proc. of ACM SOSP*. doi:10.1145/3731569.3764815
- [39] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361* (2024). <https://arxiv.org/abs/2401.14361>
- [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025). <https://arxiv.org/abs/2505.09388>
- [41] Yuchen Yang, Yaru Zhao, Pu Yang, Shaowei Wang, and Zhi-Hua Zhou. 2026. ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling. *arXiv preprint arXiv:2601.21198* (2026). <https://arxiv.org/abs/2601.21198>
- [42] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. *Annual Meeting of the Association for Computational Linguistics* (2024). doi:10.18653/v1/2024.acl-long.623
- [43] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. Flash-Infer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems*. https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html
- [44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *Proc. of USENIX OSDI*. <https://www.usenix.org/conference/osdi22/presentation/yyu>
- [45] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2026. Taming latency-memory trade-off in MoE-based LLM serving via fine-grained expert offloading. In *Proc. of ACM EuroSys*. doi:10.1145/3767295.3769319
- [46] Tianyi Zhang, Mohsen Hariri, Shaochen Zhong, Vipin Chaudhary, Yang Sui, Xia Hu, and Anshumali Shrivastava. 2025. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float. In *Annual Conference on Neural Information Processing Systems*. doi:10.52202/085713-3306
- [47] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583. doi:10.52202/079017-2000
- [48] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proc. of IEEE/ACM ICCAD*. doi:10.1145/3676536.3676741
- [49] Shuzhang Zhong, Yanfan Sun, Ling Liang, Runsheng Wang, Ru Huang, and Meng Li. 2025. HybriMoE: Hybrid CPU-GPU Scheduling and cache management for efficient MoE inference. In *Proc. of ACM/IEEE DAC*. doi:10.1109/DAC63849.2025.11133274
- [50] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proc. of USENIX OSDI*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [51] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards optimal large language model serving throughput. In *Proc. of USENIX OSDI*. <https://www.usenix.org/conference/osdi25/presentation/zhu-kan>

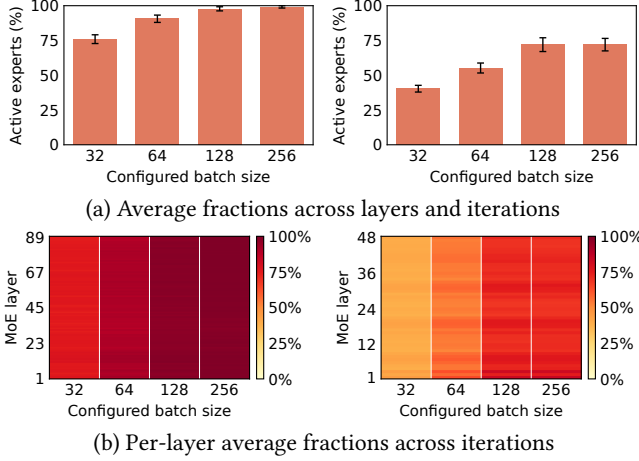


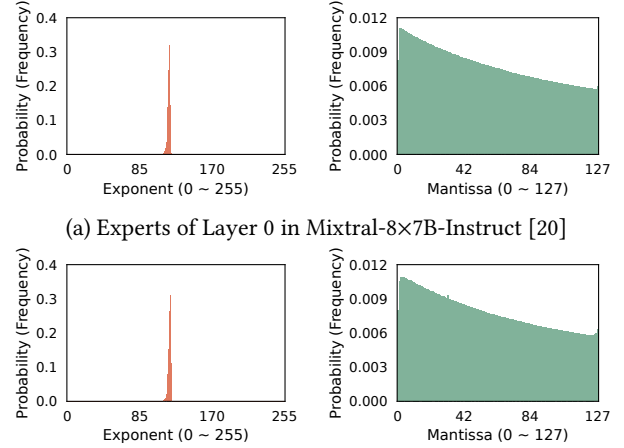
Figure 10. Active-expert fractions during decoding for GLM-4.5 [34] (left) and Qwen3-Next-80B-A3B-Instruct [40] (right).

This appendix provides analyses and evaluation details omitted from the paper. We first characterize large-batch expert activation and the bit-level compressibility of MoE expert weights (Appendix A), then supplement our experiments with additional results (Appendix B), and finally verify that FluxMoE preserves model quality (Appendix C).

A Additional Analysis

Large-batch expert activation. To quantify expert activation at large batch sizes, we run vLLM [22] with the model and prompt configurations in §6.1 and profile the fraction of routed experts activated at configured batch sizes of 32, 64, 128, and 256. We select GLM-4.5 [34] and Qwen3-Next-80B-A3B-Instruct [40] since they combine large routed-expert pools (160 and 512 experts per layer, respectively) with sparse per-token routing (top-8 and top-10, respectively). For every decoding iteration and MoE layer, we record the number of distinct routed experts selected by the scheduled tokens. Shared experts are excluded from both the counts and pool sizes. Since KV capacity may prevent the system from processing the full configured batch, we measure activation only in decoding iterations whose scheduled sequence count reaches at least 90% of the run’s maximum. Each retained layer–iteration pair forms one sample, whose active-expert fraction is the distinct routed-expert count divided by that layer’s routed-expert count.

Figure 10(a) pools these samples’ active-expert fractions across layers and iterations, reporting their average and standard deviation. The average active-expert fraction rises with configured batch size, reaching nearly 100% for GLM-4.5 [34] and over 70% for Qwen3-Next-80B-A3B-Instruct [40] at a configured batch size of 256 since larger scheduled batches generate more routing decisions. GLM-4.5 has a higher fraction since each GLM-4.5 token selects 5% of its expert pool, whereas each Qwen3-Next-80B-A3B-Instruct token selects about 2%. For Qwen3-Next-80B-A3B-Instruct, the fraction



(b) Experts of Layer 20 in Qwen3-Next-80B-A3B-Instruct [40]

Figure 11. Exponent and mantissa distribution analysis of MoE expert weights.

plateaus as the configured batch size increases from 128 to 256 since limited KV capacity prevents more sequences from being scheduled. Figure 10(b) instead averages the samples’ active-expert fractions across retained iterations separately for each layer. The active-expert fraction increases in nearly all layers rather than only a few outliers since a larger scheduled batch supplies more routing decisions to every layer. These results motivate layer-wise expert streaming: large batches activate most experts within each layer, shifting the memory-saving opportunity from expert-level caching to cross-layer memory reuse.

Expert-weight compressibility. To characterize the compressibility of MoE expert weights, we profile the BFloat16 routed-expert weights of Mixtral-8x7B-Instruct [20] and Qwen3-Next-80B-A3B-Instruct [40] across their MoE layers. We separate the weights into exponent and mantissa fields and compute their empirical distributions across experts.

Figure 11 presents one representative layer from each model. The exponent bits (8 bits in BFloat16) are highly concentrated within a narrow range of magnitudes, exhibiting a sharp probability peak and correspondingly low entropy. In contrast, the mantissa fields (7 bits) are nearly uniformly distributed, suggesting limited compression gains from their empirical symbol frequencies. This contrast in field-level entropy is consistent across the profiled models and layers. Note that prior studies [9, 15, 16, 46] have also identified the distinct entropy patterns in mantissas and exponents in traditional dense models. Our profiling targets the sparse computing paradigm in MoE and confirms that MoE expert weights show a highly compressible exponent structure.

B Additional Evaluation Results

We supplement §6 with four sets of results: (i) offline and online performance under TP-only FluxMoE execution (Figures 12 and 13); (ii) intermediate-batch materialization mea-

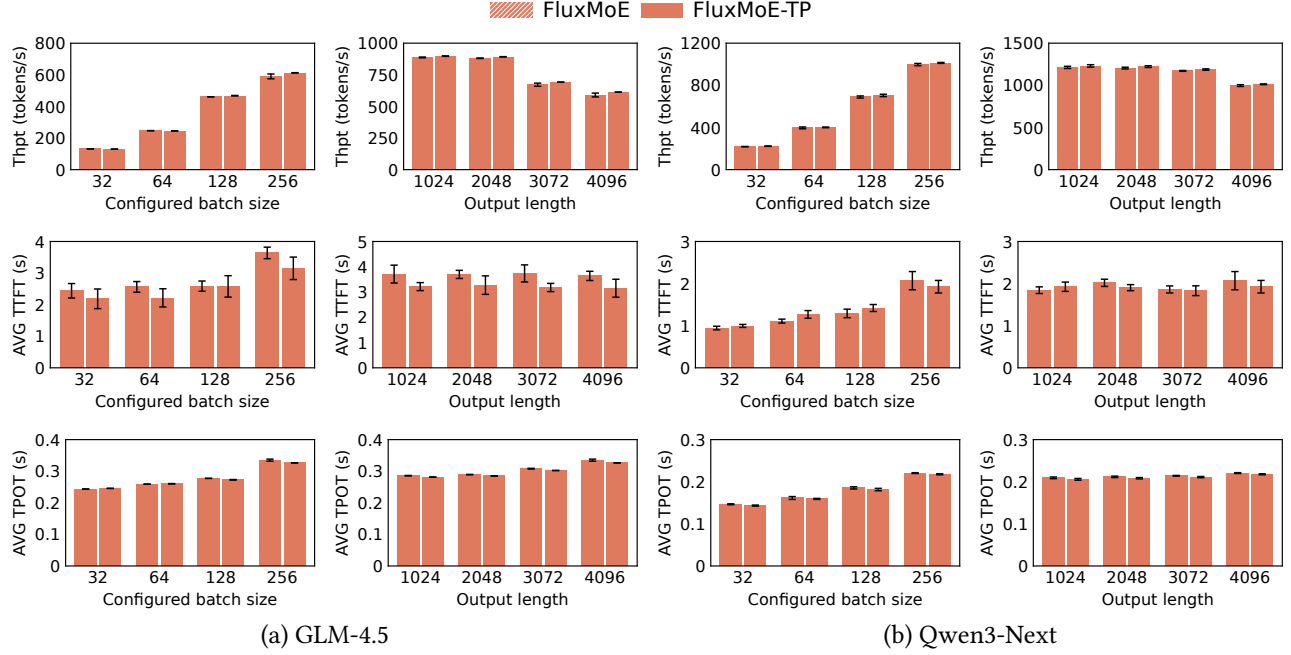


Figure 12. Offline performance of FluxMoE and FluxMoE-TP.

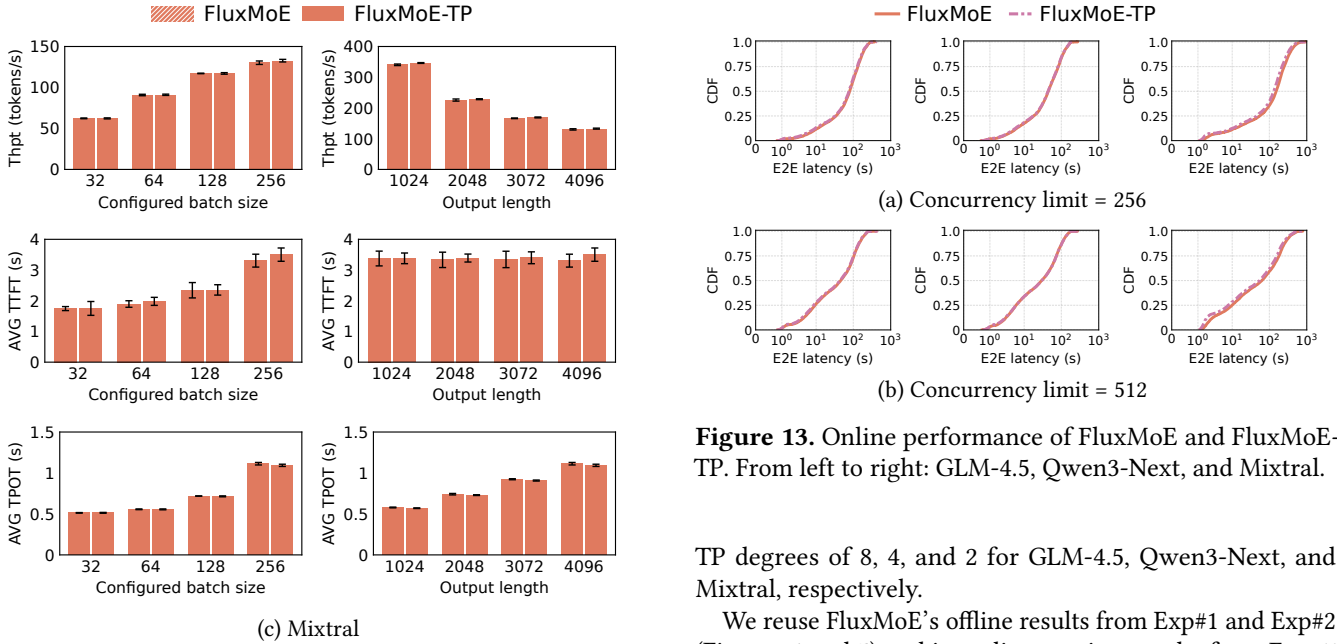


Figure 13. Online performance of FluxMoE and FluxMoE-TP. From left to right: GLM-4.5, Qwen3-Next, and Mixtral.

Figure 12. Offline performance of FluxMoE and FluxMoE-TP (continued).

surements for Exp#4 (Table 5); (iii) planner-sensitivity data for Exp#5 (Table 6); and (iv) online serving results at additional request rates for Exp#7 (Figure 14).

FluxMoE-TP offline and online performance. Because KTransformers [7] does not support EP, we implement a TP-only variant, FluxMoE-TP, retaining the mechanisms in §4. We evaluate it offline and online with the same GPU deployments and residency configurations as FluxMoE, using

TP degrees of 8, 4, and 2 for GLM-4.5, Qwen3-Next, and Mixtral, respectively.

We reuse FluxMoE’s offline results from Exp#1 and Exp#2 (Figures 4 and 5) and its online serving results from Exp#7 (Figure 9). Figures 12 and 13 compare them with FluxMoE-TP. In offline evaluations, FluxMoE-TP stays within 3.9% of FluxMoE in throughput and 2.6% in average TPOT. Across all evaluated model-workload settings, FluxMoE-TP reduces FluxMoE’s mean TTFT by at most 14.9%. For online serving evaluations, FluxMoE-TP’s P50 and P95 E2E latencies differ from FluxMoE’s by at most 4.8% and 3.8%, respectively, on GLM-4.5 and Qwen3-Next. On Mixtral, FluxMoE-TP reduces P50 by 13.6% (14.4%) and P95 by 12.0% (6.7%) at concurrency limit 256 (512). The similar offline and online performance

Table 5. (Exp#4) Additional expert-materialization results at $B_{\text{cfg}} \in \{64, 128\}$ and at output length 4,096 (mean $\pm$ std., ms). CPU Offloading and Decompression report isolated PCIe-transfer and decompression time, respectively. E2E materialization spans load issuance to tensor readiness, including queueing, stream waits, and launch delay; Stall-free window spans issuance until tensor use; AVG stall is the remaining wait, averaged across layers. For FluxMoE-H, C and D denote CPU- and compressed-GPU-backed layers, respectively.

Model	Method	B_{cfg}	CPU Offloading	Decompression	E2E materialization	Stall-free window	AVG stall
GLM-4.5	FluxMoE	64	–	2.17 ± 0.01	2.17 ± 0.01	5.37 ± 0.54	< 0.01
	FluxMoE	128	–	2.17 ± 0.01	2.17 ± 0.01	5.68 ± 0.90	< 0.01
Mixtral	FluxMoE	64	15.28 ± 4.97	3.13 ± 0.15	27.84 ± 1.66	21.64 ± 6.50	7.17 ± 4.76
	FluxMoE-H	64	155.02 ± 16.82	3.51 ± 0.02	C: 207.96 ± 27.75 D: 3.51 ± 0.02	C: 106.50 ± 75.59 D: 12.65 ± 13.77	13.00 ± 39.19
	FluxMoE	128	15.30 ± 4.91	3.13 ± 0.15	28.08 ± 1.49	20.86 ± 6.68	7.85 ± 4.93
	FluxMoE-H	128	146.10 ± 13.41	3.51 ± 0.01	C: 206.70 ± 29.06 D: 3.52 ± 0.02	C: 106.37 ± 72.60 D: 13.31 ± 15.69	13.05 ± 38.17

Table 6. (Exp#5) Released down-projection tensors and GPU memory across all GPUs. Each cell reports released storage / total compressed down-projection storage; fused gate/up-projection tensors remain untouched.

Setting		GLM-4.5		Qwen3-Next		Mixtral	
		Tensors (released/total)	Memory (GB) (released/total)	Tensors (released/total)	Memory (GB) (released/total)	Tensors (released/total)	Memory (GB) (released/total)
All-compressed		0/14,240	0/151.2	0/24,576	0/35.0	0/256	0/20.3
AG=0.25	CI=8	9,200/14,240	97.7/151.2	9,216/24,576	13.1/35.0	42/256	3.3/20.3
AG=0.25	CI=16	4,600/14,240	48.8/151.2	4,864/24,576	6.9/35.0	24/256	1.9/20.3
AG=0.25	CI=32	2,320/14,240	24.6/151.2	2,432/24,576	3.5/35.0	18/256	1.4/20.3
AG=0.25	CI=64	1,200/14,240	12.7/151.2	1,280/24,576	1.8/35.0	14/256	1.1/20.3
CI=16	AG=0.25	4,600/14,240	48.8/151.2	4,864/24,576	6.9/35.0	24/256	1.9/20.3
CI=16	AG=0.50	9,200/14,240	97.7/151.2	9,216/24,576	13.1/35.0	40/256	3.2/20.3
CI=16	AG=0.75	9,960/14,240	105.7/151.2	9,600/24,576	13.7/35.0	54/256	4.3/20.3
CI=16	AG=1.00	11,680/14,240	124.0/151.2	9,728/24,576	13.8/35.0	72/256	5.7/20.3

of FluxMoE and FluxMoE-TP shows that FluxMoE’s gains over KTransformers do not come from enabling EP.

(Exp#4) Materialization across batch sizes. Table 5 extends the per-layer materialization breakdown in Table 4 to the intermediate $B_{\text{cfg}} = 64$ and 128 settings, omitted from the main paper for space. At both intermediate batches, GLM-4.5 exposes less than 0.01 ms of average stall, consistent with the $B_{\text{cfg}} = 32$ and 256 results shown in Table 4. Across all four batches, GLM-4.5’s E2E materialization remains 2.17 ms, while Mixtral’s varies by at most 1.1%, showing that increasing batch size does not systematically slow materialization. At $B_{\text{cfg}} = 64$ and 128, FluxMoE reduces Mixtral’s average stall by 39.8–44.8% relative to FluxMoE-H, consistent with the stall reductions at $B_{\text{cfg}} = 32$ and 256 in Table 4.

(Exp#5) Tensor release across CI and AG settings. Table 6 reports released tensor counts and storage volumes for all CI and AG settings, extending the default CI=16 and AG=0.25 results in §6.4. All runs stop before exhausting the down-projection tensors, so these results cover only the planner’s first tensor-level stage; no fused gate/up-projection tensor is

offloaded. At CI=16 and AG=0.25, the planner releases 4,600, 4,864, and 24 down-projection tensors for GLM-4.5, Qwen3-Next, and Mixtral, respectively. This frees 48.8 GB, 6.9 GB, and 1.9 GB of GPU memory, equivalent to 32.3%, 19.7%, and 9.4% of their compressed down-projection storage. Relative to each model’s total compressed routed-expert storage, the released fractions are 10.8%, 6.6%, and 3.1%, respectively. The released fractions differ across models since the planner stops offloading when additional loading would delay inference, rather than always targeting a fixed release fraction. GLM-4.5 permits more adjustments since H20 provides higher PCIe bandwidth. For Mixtral, $L_r = 4$, so each of the 12 AG=0.25 adjustments offloads one down-projection tensor per GPU rank, releasing 24 tensors across two ranks.

At AG=0.25, increasing CI from 8 to 64 reduces released GPU memory by 87.0%, 86.3%, and 66.7% for GLM-4.5, Qwen3-Next, and Mixtral, respectively. Released GPU memory decreases across all three models since a larger CI invokes the planner less often, allowing fewer reclamation steps before the workload completes.

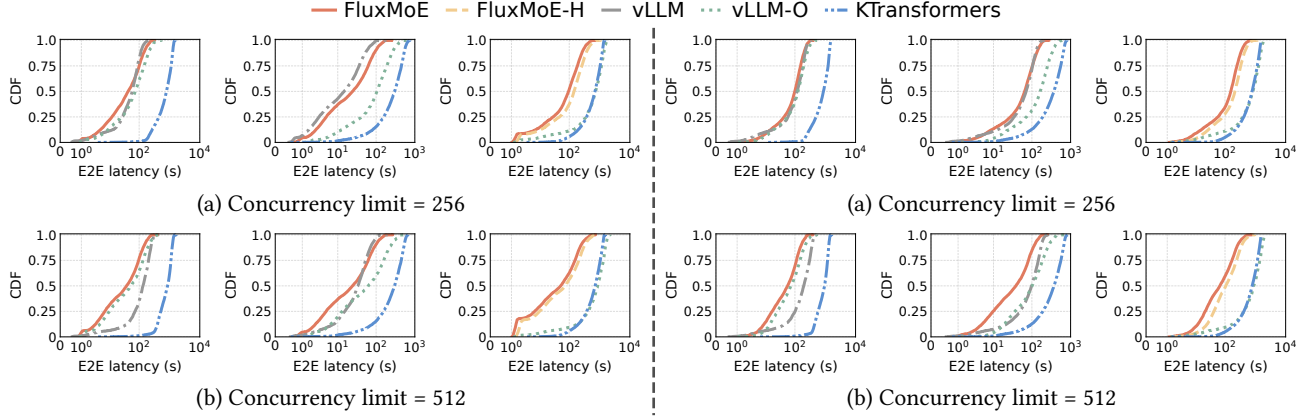


Figure 14. (Exp#7) Online performance at 5 requests/s (left) and 20 requests/s (right). For each panel, from left to right: GLM-4.5, Qwen3-Next, and Mixtral.

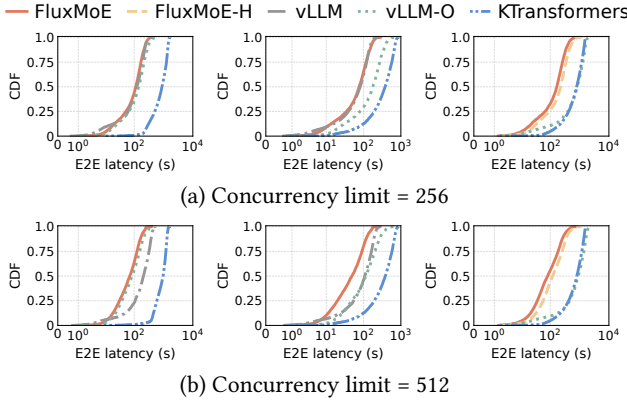


Figure 14. (Exp#7) Online performance at 40 requests/s. From left to right: GLM-4.5, Qwen3-Next, and Mixtral.

At CI=16, increasing AG from 0.25 to 1.0 raises the released fraction of compressed down-projection storage from 32.3% to 82.0% for GLM-4.5, 19.7% to 39.4% for Qwen3-Next, and 9.4% to 28.1% for Mixtral. The total released tensor count does not scale linearly with AG: a larger AG offloads more tensors per reclamation but also increases materialization demand, causing the planner to trigger fewer reclamations before completion. For GLM-4.5 and Qwen3-Next, increasing AG from 0.25 to 0.5 doubles the per-step tensor count. Since the number of adjustments changes little, the total released tensor count nearly doubles. Above AG=0.5, larger per-step releases increase materialization time and cause the planner to stop offloading earlier, so total release grows more slowly. Mixtral’s released fraction grows nearly linearly with AG. Each compressed tensor in Mixtral is large, so increasing AG substantially increases the memory released per adjustment, while the number of adjustments decreases only from 12 at AG=0.25 to 9 at AG=1.0.

(Exp#7) Online serving at additional request rates. We report all three models at 10 requests/s in §6.5; Figure 14 extends the evaluation to 5, 20, and 40 requests/s to cover

lighter and heavier loads. At 5 requests/s, FluxMoE on GLM-4.5 is saturated at a concurrency limit of 256 but unsaturated at 512. On Qwen3-Next, both FluxMoE and vLLM are unsaturated under both limits. All remaining settings are saturated.

At 5 requests/s, FluxMoE’s P95 latency on Qwen3-Next is 66.0% and 30.5% higher than vLLM’s at limits of 256 and 512, respectively. In these runs, B_{act} is limited by ready requests rather than KV capacity, so FluxMoE gains no batching benefit to offset its materialization overhead. For GLM-4.5 at a concurrency limit of 256, FluxMoE’s P95 is 39.0% higher than vLLM’s. The reason is that the 256 limit prevents FluxMoE from using its additional KV capacity to increase $\bar{B}_{\text{act}}$, but it still incurs per-layer materialization overhead. At a limit of 512, FluxMoE’s P95 is 33.7% lower than vLLM’s. Raising the limit from 256 to 512 allows FluxMoE to exploit its additional KV capacity, sustain a larger $\bar{B}_{\text{act}}$ than vLLM, and reduce queueing. For Mixtral, FluxMoE lowers P95 relative to FluxMoE-H by 27.5% and 22.4% at limits of 256 and 512, respectively, since bandwidth-balanced backends reduce materialization overhead under the same offload budget.

At 20 and 40 requests/s, every system saturates for all three models under both concurrency limits. FluxMoE achieves the lowest P50 in every case. For instance, at a concurrency limit of 256 and 20 requests/s, FluxMoE reduces P50 by 15.5% over vLLM on GLM-4.5, 8.7% over vLLM on Qwen3-Next, and 26.8% over FluxMoE-H on Mixtral. At 40 requests/s, the corresponding reductions are 8.6%, 6.8%, and 25.5%, respectively. For GLM-4.5 and Qwen3-Next, additional KV capacity supports larger B_{act} than vLLM. For Mixtral, FluxMoE’s gains over FluxMoE-H instead reflect lower materialization overhead at the same KV capacity.

Overall, these additional request rates contextualize the 10 requests/s results in §6.5: extra KV capacity benefits serving when it can increase B_{act} ; limited request availability or a restrictive concurrency limit can leave that capacity unused while materialization overhead remains.

Table 7. (Exp#8) Perplexity evaluation of the three models on the WikiText-2-Raw-v1 [26] test split.

Model	Tokenizer	Scored tokens	Number of windows	TP/EP	Perplexity		
					vLLM	FluxMoE	$ \Delta $
GLM-4.5	PreTrainedTokenizerFast	289,722	563	8/8	3.6497	3.6497	0.0000
Qwen3-Next	Qwen2Tokenizer	299,077	582	4/4	5.2456	5.2456	0.0000
Mixtral	LlamaTokenizerFast	334,659	651	4/4	3.6976	3.6976	0.0000

C Model-Quality Evaluation

We evaluate whether FluxMoE’s dynamic materialization and lossless compression affect model quality.

(Exp#8) Perplexity evaluation. Following prior work on LLM compression [12, 24], we use perplexity on the complete test split of WikiText-2-Raw-v1 [26] to evaluate whether FluxMoE preserves the model quality of weight-resident vLLM [22]. We evaluate the same three MoE models used in our performance experiments: GLM-4.5, Qwen3-Next, and Mixtral. For each model, both systems use the same checkpoint, tokenizer, TP/EP configuration, and evaluation inputs. For Mixtral, we use TP/EP degrees of 4/4 since weight-resident vLLM [22] runs out of GPU memory at a TP degree of 2 on L40S; TP=4 allows both systems to complete the perplexity comparison using the same checkpoint. To construct the evaluation input, we concatenate all text entries in the test split using double-newline separators and tokenize the resulting text as a single continuous sequence with each model’s native tokenizer. We do not add beginning-of-sequence (BOS) or end-of-sequence (EOS) tokens. We use a sliding context window of 2,048 tokens with a stride of 512 tokens. Overlapping tokens provide context only; for each window, we score only the newly introduced tokens. We process each window independently with a batch size of one, without cross-window KV cache reuse or prefix caching.

Table 7 compares the perplexity obtained by vLLM and FluxMoE. Since each model uses its native tokenizer, the same WikiText-2 input produces different numbers of scored tokens and sliding windows. vLLM and FluxMoE achieve identical perplexities for all three models: 3.6497 for GLM-4.5, 5.2456 for Qwen3-Next, and 3.6976 for Mixtral, demonstrating that FluxMoE’s dynamic expert materialization and lossless decompression introduce no measurable model-quality loss relative to vLLM at the reported precision.